

Percorso completo per creare un tactical multiplayer a turni simultanei con Unreal Engine

Sintesi esecutiva

L'obiettivo del corso è costruire **un vertical slice giocabile**, ispirato alla struttura tattica di *Atlas Reactor* ma con identità, personaggi, mappe, nomi e asset originali. Il riferimento meccanico è il ciclo nel quale tutti i giocatori pianificano contemporaneamente e il server risolve le azioni secondo l'ordine **Prep → Dash → Blast → Move**. La guida originale di *Atlas Reactor* descrive infatti una modalità temporizzata di decisione, seguita dalla risoluzione comune delle azioni nelle quattro fasi. ¹

Il progetto didattico proposto, chiamato **ReactorTactics**, sarà intenzionalmente più piccolo:

Decisione	Scelta per il corso
Modalità	Duello uno contro uno
Squadre	Due unità per giocatore
Mappa	Griglia quadrata dodici per dodici
Abilità	Tre per unità: scudo, dash, attacco
Turno	Pianificazione simultanea e risoluzione per fasi
Multiplayer iniziale	Listen server e rete locale
Multiplayer successivo	Sessioni online ed Epic Online Services
Grafica	Primitive, materiali semplici e asset gratuiti
Durata prevista	Circa cinquantotto ore, distribuite su otto settimane
Linguaggi	C++ per regole e networking, Blueprint per presentazione e contenuti

Alla data del primo agosto 2026, la versione corrente raccomandata per il corso è **Unreal Engine 5.8.1**: Unreal Engine 5.8 è stato pubblicato il 17 giugno 2026, mentre l'hotfix 5.8.1 è arrivato il 28 luglio con oltre 260 correzioni. Conviene fissare questa versione nel `README.md` e non aggiornarla durante il corso, salvo bug bloccanti. ²

Il risultato finale non sarà ancora un prodotto live commerciale completo. Sarà però un prototipo robusto con:

- selezione delle unità e telecamera tattica;
- griglia logica separata dalla grafica;
- abilità configurabili tramite Data Asset;
- pianificazione nascosta all'avversario;
- risoluzione deterministica sul server;
- movimento, dash, danno, scudi, linea di vista e collisioni;

- replica multiplayer;
- test automatici;
- build Windows eseguibile.

L'approccio consigliato è **C++ sotto, Blueprint sopra**. Epic considera normale combinare i due sistemi: il C++ è adatto alle fondamenta, alle regole condivise e alle API riutilizzabili, mentre Blueprint consente di iterare rapidamente su UI, effetti, asset e comportamento di presentazione. ³

Ambito, pubblico e prerequisiti

Il corso è progettato per uno sviluppatore che conosce C# e i concetti di programmazione a oggetti, ma non ha ancora esperienza significativa con il C++ di Unreal. Non è necessario conoscere il C++ moderno in profondità prima di iniziare: i concetti vengono introdotti nel momento in cui servono.

La guida ufficiale Epic dedicata a chi proviene da Unity è particolarmente utile perché traduce concetti familiari come componenti, scene, script e prefab nell'ecosistema Unreal. ⁴

La mappa mentale iniziale sarà questa:

Concetto C# o Unity	Equivalente tipico in Unreal
<code>class</code>	Classe C++ con <code>UCLASS()</code>
Proprietà serializzata	<code>UPROPERTY()</code>
Metodo visibile nell'editor	<code>UFUNCTION(BlueprintCallable)</code>
<code>List<T></code>	<code>TArray<T></code>
<code>Dictionary<K,V></code>	<code>TMap<K,V></code>
<code>GameObject</code>	<code>AActor</code>
<code>MonoBehaviour</code>	<code>AActor</code> , <code>APawn</code> o <code>UActorComponent</code>
Prefab	Blueprint Class
ScriptableObject	<code>UDataAsset</code> o <code>UPrimaryDataAsset</code>
Evento C#	Delegate Unreal
<code>null</code>	<code>nullptr</code>
Garbage collector .NET	Garbage collector degli <code>UObject</code> , con regole Unreal

Gli obiettivi di apprendimento sono cinque. Al termine del percorso dovresti saper:

1. orientarti nell'Editor, nel Content Browser, nella compilazione e nel flusso Play In Editor;
2. creare classi Unreal in C++ usando reflection, `UCLASS`, `UPROPERTY`, `UFUNCTION`, `TArray` e `TObjectPtr`;
3. progettare regole tattiche indipendenti dall'animazione;
4. implementare un modello multiplayer server-authoritative;
5. testare, debuggare e produrre una build distribuibile.

Assunzioni di progetto

Poiché budget, piattaforme e portata commerciale non sono ancora specificati, il corso assume:

Vincolo	Assunzione
Budget	Da zero a basso
Sistema di sviluppo	Windows undici
Piattaforma finale iniziale	PC Windows
Input	Mouse e tastiera
Account online	Non richiesti per il primo prototipo
Art team	Nessuno
Backend dedicato	Non disponibile all'inizio
Matchmaking	Fuori dallo scope del primo MVP
Persistenza	Solo configurazioni locali
Anti-cheat	Validazione server di base
Target grafico	Hardware PC medio, senza ray tracing obbligatorio

Hardware consigliato

La documentazione corrente di Unreal Engine raccomanda per lo sviluppo su Windows un processore quad-core da almeno 2,5 GHz, 32 GB di RAM e una GPU DirectX 12 con almeno 8 GB di memoria video. Per lavorare in modo confortevole, soprattutto durante compilazione degli shader e del C++, sono preferibili più core e un SSD NVMe. ⁵

Configurazione pratica:

Componente	Minimo praticabile per il corso	Consigliato
CPU	Sei core	Dodici o più core
RAM	Sedici GB	Trentadue GB
GPU	Quattro-sei GB VRAM	Otto GB VRAM
Disco libero	Centoventi GB	Duecento GB
Risoluzione	1920×1080	2560×1440
Connessione	Necessaria per download	Banda larga stabile

Sedici GB possono funzionare per il prototipo, ma Unreal Editor, Visual Studio, browser e compilazione shader possono saturarli rapidamente. I simboli di debug dell'Editor richiedono parecchio spazio, quindi vanno installati soltanto se il disco lo consente.

Installazione dell'ambiente

Scaricamenti principali

- [Epic Games Launcher e Unreal Engine](#)
- [Visual Studio Community](#)
- [Git](#)
- [Git LFS](#)

Unreal Engine viene normalmente installato dal pannello **Unreal Engine → Library** dell'Epic Games Launcher, dove è possibile aggiungere una versione del motore, selezionare i componenti e applicare gli hotfix disponibili. ⁶

Procedura consigliata:

1. Installa Epic Games Launcher.
2. Accedi con un account Epic.
3. Apri **Unreal Engine → Library**.
4. Aggiungi una nuova installazione.
5. Seleziona **Unreal Engine 5.8.1**.
6. In **Options**, mantieni:
7. Core Components;
8. Starter Content;
9. Templates and Feature Packs;
10. Engine Source.
11. Aggiungi **Editor Symbols for Debugging** se hai spazio sufficiente e desideri eseguire debug più profondo di Blueprint e codice motore.
12. Puoi rimuovere Android, Linux e piattaforme console per ridurre l'installazione.

I simboli dell'Editor sono richiesti per alcune funzioni avanzate del debugger Blueprint integrato in Visual Studio. ⁷

Configurazione di Visual Studio

Installa **Visual Studio 2026 Community**. Nel Visual Studio Installer seleziona il workload:

Game development with C++

All'interno dei componenti opzionali seleziona:

Visual Studio Tools for Unreal Engine
Visual Studio debugger tools for Unreal Engine Blueprints
Unreal Engine Test Adapter
MSVC C++ build tools

Windows 11 SDK
C++ profiling tools

Microsoft indica espressamente il workload **Game development with C++** e gli strumenti Unreal come configurazione prevista per questo tipo di sviluppo. ⁸

Dopo la creazione del progetto:

1. Apri il file `.uproject` con Visual Studio.
2. Vai in **Project → Configure Tools for Unreal Engine**.
3. Verifica che l'integrazione Unreal, il supporto Blueprint e il Test Adapter risultino configurati.
4. In Unreal Editor apri **Edit → Editor Preferences → Source Code** e seleziona Visual Studio.

Se possiedi già Visual Studio 2022 recente, può ancora essere sufficiente per molte configurazioni Unreal; il corso utilizza però Visual Studio 2026 per restare allineato alla toolchain corrente.

Struttura delle cartelle

Crea il progetto in un percorso corto e senza spazi strani:

`C:\Dev\ReactorTactics`

La struttura prevista sarà:

```
ReactorTactics/
├── Config/
├── Content/
│   ├── ReactorTactics/
│   │   ├── Abilities/
│   │   ├── Blueprints/
│   │   ├── Input/
│   │   ├── Maps/
│   │   ├── Materials/
│   │   ├── UI/
│   │   └── VFX/
│   └── Source/
│       ├── ReactorTactics/
│       │   ├── Abilities/
│       │   ├── Combat/
│       │   ├── Core/
│       │   ├── Game/
│       │   ├── Grid/
│       │   ├── Player/
│       │   ├── Tests/
│       │   ├── Turn/
│       │   └── Units/
```

```
├─ ReactorTactics.uproject
└─ README.md
```

Non versionare queste directory:

```
Binaries/
DerivedDataCache/
Intermediate/
Saved/
.vs/
```

Per asset binari come `.uasset` e `.umap`, Git LFS evita che la cronologia Git cresca in modo ingestibile:

```
git lfs install
git lfs track "*.uasset"
git lfs track "*.umap"
git add .gitattributes
```

Architettura tecnica e decisioni

Distribuzione tra C++ e Blueprint

La divisione proposta è circa **sessanta per cento C++ e quaranta per cento Blueprint**, misurata in responsabilità e non in righe di codice.

C++	Blueprint
Coordinate della griglia	Aspetto delle celle
Stato del turno	Widget del timer
Validazione delle azioni	Preview degli attacchi
Risoluzione simultanea	Animazioni
Danno e stati	Niagara e audio
Replica e RPC	Materiali evidenziati
Test automatici	Layout della HUD
Data contract	Configurazione delle unità

Il principio è semplice: **una regola che deve essere identica sul server e su tutti i client vive in C++**; un elemento che cambia frequentemente per motivi estetici o di UX vive in Blueprint.

Unreal permette di esporre classi, funzioni e proprietà C++ all'Editor attraverso il sistema di reflection basato sulle macro `UCLASS`, `UFUNCTION` e `UPROPERTY`. ⁹

Uso di C# tramite plugin

Esistono plugin non ufficiali che permettono di utilizzare .NET e C# in Unreal. Tra questi, UnrealSharp integra C# e .NET nel motore, mentre UnrealCLR offre un runtime .NET integrato. ¹⁰

Per questo corso **non sono raccomandati nel nucleo del progetto**. Le ragioni sono pratiche:

- aggiungono una dipendenza esterna alla versione del motore;
- complicano packaging, debugging e aggiornamenti;
- la maggior parte della documentazione, degli esempi e delle API di gameplay Unreal è pensata per C++ e Blueprint;
- eliminerebbero proprio l'occasione di imparare il modello nativo di Unreal.

Possono diventare un esperimento successivo al vertical slice, per esempio implementando un tool editoriale in C#. Non dovrebbero però controllare il resolver, la replica o lo stato della partita.

Gameplay Ability System

Il Gameplay Ability System di Epic è progettato per abilità, attributi, effetti, cooldown e scenari multiplayer complessi. È una soluzione potente e adatta anche a giochi con set di abilità articolati. ¹¹

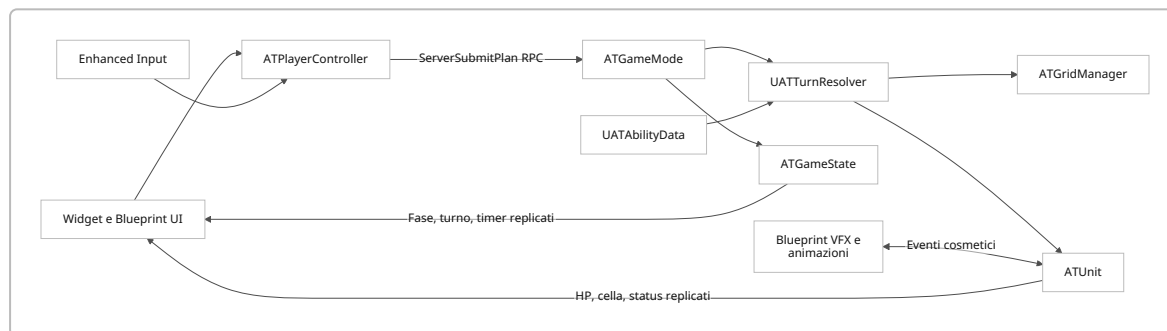
Per il primo corso è però eccessivo. Introdurrebbe contemporaneamente:

- Ability System Component;
- Gameplay Tags;
- Gameplay Effects;
- Attribute Sets;
- prediction keys;
- execution calculations;
- replication policies.

Il vertical slice userà quindi un sistema leggero basato su `UPrimaryDataAsset`. I Primary Data Asset possono essere creati direttamente nell'Editor e gestiti dall'Asset Manager, risultando adatti a descrivere contenuti configurabili. ¹²

GAS diventa un modulo avanzato successivo quando il gioco avrà più personaggi, stacking di effetti, status complessi e molte abilità condivise.

Architettura delle classi



Responsabilità principali:

Classe	Responsabilità
AATGameMode	Regole autorevoli, timer, piani ricevuti, vittoria
AATGameState	Fase, turno, tempo e punteggio replicati
AATPlayerController	Input locale, selezione, RPC al server
AATPlayerState	Team e dati persistenti del giocatore
AATGridManager	Conversione cella/mondo, occupazione e validità
AATUnit	Salute, team, cella, abilità e stati
UATAbilityData	Definizione data-driven delle abilità
UATTurnResolver	Raccolta ed applicazione deterministica degli effetti
UATCombatMath	Funzioni pure e testabili di danno
Widget Blueprint	Presentazione, input UI e feedback

In Unreal, GameMode e GameState hanno ruoli distinti: il GameMode controlla le regole autorevoli della partita, mentre il GameState rappresenta informazioni di partita che devono essere disponibili ai client.

13

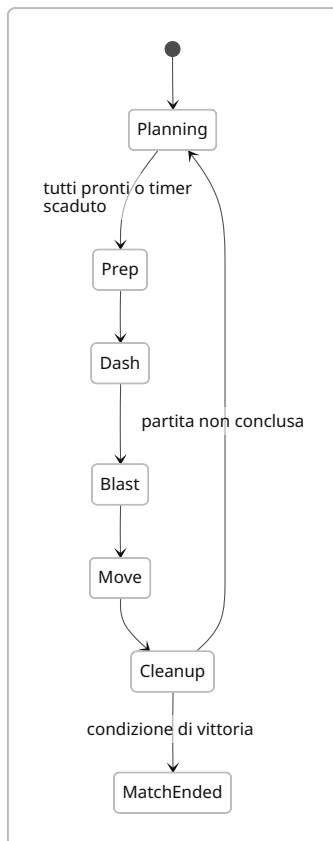
Flusso del corso



Meccaniche e modello multiplayer

Ciclo del turno

Il ciclo del prototipo sarà:



Durante **Planning**, ogni giocatore seleziona per ciascuna unità:

Una abilità opzionale
 Un bersaglio o una cella
 Una destinazione di movimento
 Conferma del piano

Durante la risoluzione:

Fase	Effetti
Prep	Scudi, buff, trappole
Dash	Spostamenti evasivi e cariche
Blast	Attacchi, cure e danni
Move	Riposizionamento ordinario
Cleanup	Cooldown, morte, punteggio, fine turno

La guida originale distingue Decision Mode e Resolution Mode e specifica l'ordine Prep, Dash, Blast e Move. Il nostro prototipo conserva la struttura, ma riduce fortemente personaggi, stati e regole. ¹⁴

Regole semplificate del vertical slice

Le decisioni iniziali saranno:

Regola	Implementazione
Durata pianificazione	Trenta secondi
Dimensione cella	Cento centimetri Unreal
Movimento normale	Quattro celle
Dash	Tre celle
Salute	Cento
Scudo base	Venti
Attacco base	Trenta danni
Occupazione	Massimo una unità per cella
Destinazione contesa	Entrambe le unità restano ferme
Danno simultaneo	Tutti gli attacchi validi della fase si applicano
Morte	Determinata dopo l'intero batch Blast
Vittoria	Prima squadra a tre eliminazioni o miglior punteggio dopo dieci turni

Il punto chiave è la distinzione tra **calcolo** e **presentazione**. Gli effetti vengono prima calcolati in batch, poi applicati. Le animazioni possono essere riprodotte una dopo l'altra per leggibilità, senza modificare il risultato tattico.

Esempio:

Unità A ha 20 HP e spara a B.
Unità B ha 20 HP e spara ad A.

Il resolver raccoglie entrambi gli attacchi.
Entrambi i danni vengono applicati.
A e B vengono eliminate.

Non bisogna quindi eliminare immediatamente A durante la prima animazione e cancellarne l'azione: l'ordine visivo non deve cambiare il risultato.

Griglia quadrata, esagonale o movimento libero

Sistema	Vantaggi	Svantaggi
Quadrato	Coordinate semplici, debug facile, A* immediato	Distanze diagonali meno uniformi
Esagonale	Distanze radiali più naturali	Matematica e authoring più complessi
NavMesh libero	Movimento fluido e mappe organiche	Collisioni simultanee più difficili

Sistema	Vantaggi	Svantaggi
Waypoint continui	Molto vicino al feeling originale	Scope elevato per un primo progetto

La scelta del corso è la **griglia quadrata logica**, anche se la grafica non dovrà mostrare necessariamente ogni casella. In futuro potrà essere sostituita da una navigazione continua mantenendo invariati GameMode, resolver e dati delle abilità.

Modello di networking

Il gioco utilizzerà un modello **server-authoritative**:

1. il client crea una preview locale;
2. il client invia il piano con una Server RPC affidabile;
3. il server verifica proprietà dell'unità, fase, cooldown, range e bersaglio;
4. il server memorizza il piano;
5. quando tutti sono pronti o scade il timer, il server risolve;
6. il server aggiorna proprietà replicate;
7. i client riproducono il risultato ricevuto.

La documentazione Unreal raccomanda che variabili importanti per il gameplay vengano modificate dall'autorità di rete e replicate ai client, invece di permettere al client di stabilirne direttamente il valore.

15

Non useremo un lockstep peer-to-peer. Anche se il gioco è a turni, un client non deve poter dichiarare:

Ho colpito.
Ho abbastanza range.
La mia unità ha ancora questa abilità.
Sono arrivato in quella cella.

Deve soltanto dichiarare:

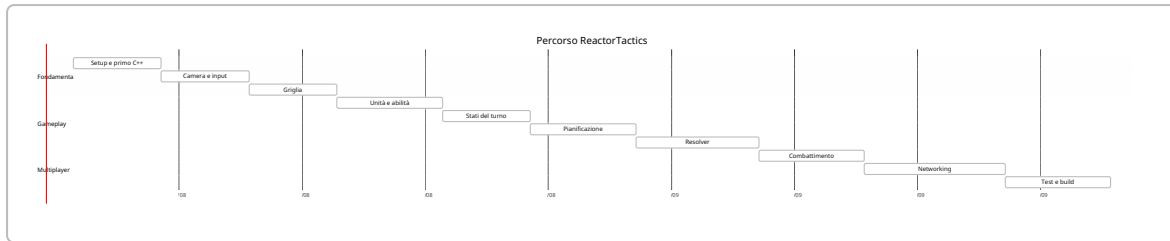
Vorrei usare questa abilità verso questa cella.

Il server decide se la richiesta è valida.

Le intenzioni nemiche non saranno replicate durante Planning. Lo stato pubblico, come fase, timer, HP e posizione corrente, sarà nel GameState o nelle unità replicate. I piani resteranno nel GameMode, che esiste soltanto sul server.

Sequenza delle lezioni

Il calendario seguente assume sei-otto ore settimanali.



Lezione uno — Configurazione e primo Actor C++

Voce	Contenuto
Obiettivo	Creare, compilare, eseguire e versionare un progetto C++
Tempo	Quattro ore
Input	UE 5.8.1, Visual Studio, Git
Output	Progetto apribile e primo log C++
File modificati	<code>.uproject</code> , <code>ATDebugActor.h</code> , <code>ATDebugActor.cpp</code> , <code>.gitignore</code> , <code>README.md</code>
Comandi chiave	<code>git init</code> , <code>git add .</code> , <code>git commit</code> , <code>Ctrl+Shift+B</code> , <code>Ctrl+Alt+F11</code>

Unreal consente di creare classi C++ dall'Editor e di renderle immediatamente disponibili come base per Blueprint. Live Coding permette inoltre di ricompilare molte modifiche durante una sessione dell'Editor.

16

Passi

1. Avvia Unreal Engine.
2. Seleziona **Games** → **Blank**.
3. Imposta:
4. C++;
5. Desktop;
6. Scalable;
7. Starter Content;
8. Ray Tracing disattivato.
9. Nome progetto: `ReactorTactics`.
10. Apri **Tools** → **New C++ Class**.
11. Seleziona `Actor`.
12. Chiamalo `ATDebugActor`.

Sostituisci `ATDebugActor.h` con:

```
#pragma once

#include "CoreMinimal.h"
```

```
#include "GameFramework/Actor.h"
#include "ATDebugActor.generated.h"

UCLASS()
class REACTORTACTICS_API AATDebugActor : public AActor
{
    GENERATED_BODY()

protected:
    virtual void BeginPlay() override;
};
```

Sostituisci `ATDebugActor.cpp` con:

```
#include "ATDebugActor.h"

void AATDebugActor::BeginPlay()
{
    Super::BeginPlay();

    UE_LOG(
        LogTemp,
        Display,
        TEXT("ReactorTactics avviato correttamente")
    );
}
```

Compila da Visual Studio:

Build → Build Solution

Torna nell'Editor e crea:

Content/ReactorTactics/Blueprints/BP_DebugActor

Scegli `ATDebugActor` come classe base, trascinalo nella mappa e premi Play. Apri **Window → Output Log** e cerca:

ReactorTactics avviato correttamente

Configura Git:

```
cd C:\Dev\ReactorTactics
git init
git lfs install
```

```
git lfs track "*.uasset"
git lfs track "*.umap"
git add .
git commit -m "chore: progetto Unreal iniziale"
```

Esercizio

Aggiungi al file header:

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Debug")
FString Message = TEXT("Ciao da Unreal");
```

Poi stampalo con:

```
UE_LOG(LogTemp, Display, TEXT("%s"), *Message);
```

Modifica il valore dal pannello Details del Blueprint.

Criterio di completamento

La soluzione compila senza errori, il Blueprint appare nel Content Browser e il messaggio viene visualizzato nell'Output Log.

Lezione due — Telecamera tattica, input e selezione

Voce	Contenuto
Obiettivo	Muovere una camera top-down e selezionare oggetti
Tempo	Cinque ore
Input	Progetto della lezione precedente
Output	Camera pan/zoom e click sul mondo
File modificati	ATPlayerController.*, BP_TacticalCamera, asset Enhanced Input
Comandi chiave	Play In Editor, showdebug enhancedinput

Enhanced Input è il sistema moderno di Unreal per definire azioni, mapping context e trigger di input.

17

Passi

1. Crea una nuova classe C++ basata su `PlayerController`.
2. Chiamala `ATPlayerController`.
3. Aggiungi il costruttore:

```

AATPlayerController::AATPlayerController()
{
    bShowMouseCursor = true;
    bEnableClickEvents = true;
    bEnableMouseOverEvents = true;
    DefaultMouseCursor = EMouseCursor::Default;
}

```

1. Crea un Blueprint **BP_TacticalCamera** basato su **Pawn**.
2. Aggiungi:
3. **SceneComponent** come root;
4. **SpringArm**;
5. **Camera**.
6. Configura lo Spring Arm:
7. Target Arm Length: **1800**;
8. rotazione: **Pitch -55**;
9. collision test disattivato.
10. Crea gli asset:

IA_Pan	Input Action Vector2D
IA_Zoom	Input Action Axis1D
IA_Select	Input Action Digital
IMC_Tactical	Input Mapping Context

Mapping suggeriti:

IA_Pan → W, A, S, D
 IA_Zoom → Mouse Wheel Axis
 IA_Select → Left Mouse Button

Nel Blueprint della camera, crea questa catena:

```

IA_Pan Triggered
→ Get Action Value
→ Break Vector2D
→ Get Actor Forward Vector
→ Get Actor Right Vector
→ moltiplica per CameraSpeed e Delta Seconds
→ Add Actor World Offset

```

Per lo zoom:

```
IA_Zoom Triggered
→ Get Action Value
→ SpringArm Target Arm Length
→ sottrai Value × 150
→ Clamp 700-2600
→ Set Target Arm Length
```

Per leggere il click:

```
IA_Select Started
→ Get Player Controller
→ Get Hit Result Under Cursor by Channel
→ Break Hit Result
→ Print String con il nome di Hit Actor
```

Nel GameMode Blueprint assegna:

```
Default Pawn Class = BP_TacticalCamera
Player Controller Class = ATPlayerController
```

Asset setup

Crea `L_Prototype` con:

- un Plane grande;
- alcuni Cube come ostacoli;
- una Directional Light;
- uno Sky Light;
- un Player Start.

Esercizio

Aggiungi rotazione della camera:

```
Q → Add Actor Local Rotation -15°
E → Add Actor Local Rotation +15°
```

Criterio di completamento

La camera si muove e zooma, il cursore è visibile e il nome dell'attore cliccato compare sullo schermo.

Lezione tre — Griglia logica e conversione delle coordinate

Voce	Contenuto
Obiettivo	Separare coordinate tattiche e coordinate del mondo
Tempo	Sei ore
Input	Camera e livello prototipo
Output	Griglia dodici per dodici con cella evidenziata
File modificati	ATTypes.h, ATGridManager.h, ATGridManager.cpp, BP_CellHighlight
Comandi chiave	Live Coding, DrawDebugLine

Passi

1. Crea Source/ReactorTactics/Core/ATTypes.h.
2. Inserisci:

```
#pragma once

#include "CoreMinimal.h"
#include "ATTypes.generated.h"

USTRUCT(BlueprintType)
struct FGridCoord
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Y = 0;

    bool operator==(const FGridCoord& Other) const
    {
        return X == Other.X && Y == Other.Y;
    }
};

FORCEINLINE uint32 GetTypeHash(const FGridCoord& Coord)
{
    return HashCombine(
        GetTypeHash(Coord.X),
        GetTypeHash(Coord.Y)
    );
}
```

1. Crea un Actor C++ ATGridManager.

2. Nel file header aggiungi:

```
#include "Core/ATTypes.h"

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Grid")
int32 Width = 12;

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Grid")
int32 Height = 12;

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Grid")
float CellSize = 100.0f;

UFUNCTION(BlueprintPure, Category = "Grid")
FVector CellToWorld(FGridCoord Cell) const;

UFUNCTION(BlueprintPure, Category = "Grid")
FGridCoord WorldToCell(FVector WorldPosition) const;

UFUNCTION(BlueprintPure, Category = "Grid")
bool IsInsideGrid(FGridCoord Cell) const;
```

1. Nel `.cpp` inserisci:

```
FVector AATGridManager::CellToWorld(FGridCoord Cell) const
{
    return GetActorLocation() + FVector(
        Cell.X * CellSize,
        Cell.Y * CellSize,
        0.0f
    );
}

FGridCoord AATGridManager::WorldToCell(
    FVector WorldPosition
) const
{
    const FVector Local = WorldPosition - GetActorLocation();

    FGridCoord Result;
    Result.X = FMath::RoundToInt(Local.X / CellSize);
    Result.Y = FMath::RoundToInt(Local.Y / CellSize);
    return Result;
}

bool AATGridManager::IsInsideGrid(FGridCoord Cell) const
{
    return Cell.X >= 0
        && Cell.Y >= 0
        && Cell.X < Width
        && Cell.Y < Height;
}
```

```

        && Cell.Y < Height;
    }

```

1. Crea `BP_GridManager` come figlio della classe C++.
2. Nel Construction Script disegna le linee con `Draw Debug Line`, oppure crea un Instanced Static Mesh di piani sottili.
3. Crea `BP_CellHighlight` usando un Plane con materiale traslucido.

Blueprint del cursore:

```

Tick
→ Get Hit Result Under Cursor
→ Impact Point
→ GridManager.WorldToCell
→ IsInsideGrid
→ GridManager.CellToWorld
→ Set Actor Location di BP_CellHighlight

```

Nota progettuale

La posizione autorevole dell'unità sarà `FGridCoord`. La posizione `FVector` servirà solo per visualizzazione e animazione. Questo evita errori di precisione e rende più facile testare la risoluzione.

Esercizio

Aggiungi:

```

UPROPERTY(EditAnywhere, BlueprintReadWrite)
TSet<FGridCoord> BlockedCells;

```

e una funzione:

```

UFUNCTION(BlueprintPure)
bool IsWalkable(FGridCoord Cell) const;

```

Una cella è valida soltanto se è dentro la griglia e non è bloccata.

Criterio di completamento

Muovendo il mouse, l'evidenziatore scatta esattamente al centro delle celle e non esce dai limiti.

Lezione quattro — Unità e abilità configurabili

Voce	Contenuto
Obiettivo	Creare unità e abilità data-driven

Voce	Contenuto
Tempo	Cinque ore
Input	Griglia funzionante
Output	Due squadre e tre Data Asset abilità
File modificati	ATAbilityData.h, ATUnit.*, DA_Shield, DA_Dash, DA_Shot
Comandi chiave	Create Advanced Asset → Data Asset

Passi

1. In `ATTypes.h` aggiungi:

```
UENUM(BlueprintType)
enum class EATActionPhase : uint8
{
    Prep,
    Dash,
    Blast,
    Move
};
```

1. Crea `UATAbilityData` basato su `UPrimaryDataAsset`.

```
#pragma once

#include "CoreMinimal.h"
#include "Engine/DataAsset.h"
#include "Core/ATTypes.h"
#include "ATAbilityData.generated.h"

UCLASS(BlueprintType)
class REACTORTACTICS_API UATAbilityData
    : public UPrimaryDataAsset
{
    GENERATED_BODY()

public:
    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
    FText DisplayName;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
    EATActionPhase Phase = EATActionPhase::Blast;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
    int32 RangeCells = 4;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
```

```

    int32 Power = 20;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
    int32 CooldownTurns = 0;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly)
    TObjectPtr<UTexture2D> Icon;
};

```

1. Crea `AATUnit`.
2. Aggiungi:

```

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Unit")
int32 TeamId = 0;

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Unit")
int32 MaxHealth = 100;

UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Unit")
int32 Health = 100;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Grid")
FGridCoord GridCell;

UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Abilities")
TArray<TObjectPtr<UATAbilityData>> Abilities;

UFUNCTION(BlueprintPure)
UATAbilityData* GetAbility(int32 Index) const;

```

Implementazione:

```

UATAbilityData* AATUnit::GetAbility(int32 Index) const
{
    return Abilities.IsValidIndex(Index)
        ? Abilities[Index]
        : nullptr;
}

```

1. Crea `BP_Unit`.
2. Aggiungi un Cylinder o il Mannequin come mesh.
3. Crea due Material Instance:
4. `MI_TeamBlue`;
5. `MI_TeamOrange`.
6. Crea tre Data Asset:

```
DA_Shield  
Phase: Prep  
Range: 0  
Power: 20
```

```
DA_Dash  
Phase: Dash  
Range: 3  
Power: 0
```

```
DA_Shot  
Phase: Blast  
Range: 5  
Power: 30
```

1. Assegna i tre asset all'array `Abilities` di `BP_Unit`.
2. Posiziona quattro unità nella mappa, due per squadra.

Blueprint di selezione

Nel click del PlayerController:

```
Get Hit Result Under Cursor  
→ Cast to BP_Unit  
→ Set SelectedUnit  
→ Set Render Custom Depth true
```

Quando cambia selezione:

```
Vecchia unità → Set Render Custom Depth false  
Nuova unità → Set Render Custom Depth true
```

Esercizio

Crea `DA_HeavyShot`:

```
Phase: Blast  
Range: 3  
Power: 45  
Cooldown: 2
```

Criterio di completamento

È possibile selezionare una unità e vedere nell'Editor le sue tre abilità configurate senza modificare il codice.

Lezione cinque — Macchina degli stati del turno

Voce	Contenuto
Obiettivo	Implementare Planning, risoluzione e Cleanup
Tempo	Cinque ore
Input	Unità e abilità
Output	Fasi automatiche visibili nella HUD
File modificati	ATGameState.*, ATGameMode.*, WBP_PhaseHUD
Comandi chiave	GetWorldTimerManager, RepNotify

Passi

1. In `ATTypes.h` aggiungi:

```
UENUM(BlueprintType)
enum class EATMatchPhase : uint8
{
    Planning,
    Prep,
    Dash,
    Blast,
    Move,
    Cleanup,
    MatchEnded
};
```

1. Crea `AATGameState`, derivato da `AGameStateBase`.
2. Nel relativo header:

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(
    FATPhaseChanged,
    EATMatchPhase,
    NewPhase
);

UPROPERTY(
    ReplicatedUsing = OnRep_Phase,
    BlueprintReadOnly,
    Category = "Match"
)
EATMatchPhase Phase = EATMatchPhase::Planning;

UPROPERTY(
    Replicated,
    BlueprintReadOnly,
    Category = "Match"
```

```

)
int32 TurnNumber = 1;

UPROPERTY(
    BlueprintAssignable,
    Category = "Match"
)
FATPhaseChanged OnPhaseChanged;

UFUNCTION()
void OnRep_Phase();

void SetPhase(EATMatchPhase NewPhase);

virtual void GetLifetimeReplicatedProps(
    TArray<FLifetimeProperty>& OutLifetimeProps
) const override;

```

1. Nel `.cpp`:

```

#include "Net/UnrealNetwork.h"

void AATGameState::SetPhase(EATMatchPhase NewPhase)
{
    if (!HasAuthority())
    {
        return;
    }

    Phase = NewPhase;
    OnRep_Phase();
    ForceNetUpdate();
}

void AATGameState::OnRep_Phase()
{
    OnPhaseChanged.Broadcast(Phase);
}

void AATGameState::GetLifetimeReplicatedProps(
    TArray<FLifetimeProperty>& OutLifetimeProps
) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    DOREPLIFETIME(AATGameState, Phase);
    DOREPLIFETIME(AATGameState, TurnNumber);
}

```

1. In `AATGameMode` crea:

```

void AATGameMode::AdvancePhase()
{
    AATGameState* State = GetGameState<AATGameState>();
    if (!State)
    {
        return;
    }

    switch (State->Phase)
    {
        case EATMatchPhase::Planning:
            State->SetPhase(EATMatchPhase::Prep);
            break;

        case EATMatchPhase::Prep:
            State->SetPhase(EATMatchPhase::Dash);
            break;

        case EATMatchPhase::Dash:
            State->SetPhase(EATMatchPhase::Blast);
            break;

        case EATMatchPhase::Blast:
            State->SetPhase(EATMatchPhase::Move);
            break;

        case EATMatchPhase::Move:
            State->SetPhase(EATMatchPhase::Cleanup);
            break;

        case EATMatchPhase::Cleanup:
            State->TurnNumber++;
            State->SetPhase(EATMatchPhase::Planning);
            break;

        default:
            break;
    }
}

```

1. Per il prototipo, assegna durate provvisorie:

Planning: 30 secondi
 Prep: 1 secondo
 Dash: 1 secondo
 Blast: 2 secondi

Move: 2 secondi
Cleanup: 1 secondo

1. Crea `WBP_PhaseHUD`.
2. Aggiungi un Text Block.
3. Su `Event Construct`:

```
Get Game State  
→ Cast ATGameState  
→ Bind Event to OnPhaseChanged  
→ Convert Enum to String  
→ Set Text
```

Esercizio

Aggiungi un countdown replicato di Planning. Il timer viene calcolato dal server; il client deve soltanto mostrarlo.

Criterio di completamento

La HUD visualizza correttamente tutte le fasi e il numero del turno aumenta dopo Cleanup.

Lezione sei — Pianificazione delle azioni e interfaccia

Voce	Contenuto
Obiettivo	Creare, modificare e bloccare un piano
Tempo	Sei ore
Input	Macchina degli stati
Output	Ability bar, target preview e pulsante conferma
File modificati	<code>FATActionPlan</code> , <code>ATPlayerController.*</code> , <code>WBP_AbilityBar</code> , <code>WBP_HUD</code>
Comandi chiave	Widget Blueprint, Blueprint Interface opzionale

Passi

1. In `ATTypes.h`, dopo una forward declaration di `AATUnit`, aggiungi:

```
class AATUnit;  
  
USTRUCT(BlueprintType)  
struct FATActionPlan  
{  
    GENERATED_BODY()  
  
    UPROPERTY(BlueprintReadWrite)  
    TObjectPtr<AATUnit> Unit = nullptr;
```

```

UPROPERTY(BlueprintReadWrite)
int32 AbilityIndex = INDEX_NONE;

UPROPERTY(BlueprintReadWrite)
FGridCoord TargetCell;

UPROPERTY(BlueprintReadWrite)
TObjectPtr<AATUnit> TargetUnit = nullptr;

UPROPERTY(BlueprintReadWrite)
bool bLocked = false;
};

```

1. Nel PlayerController aggiungi:

```

UPROPERTY(BlueprintReadOnly, Category = "Planning")
TArray<FATAActionPlan> LocalPlans;

UPROPERTY(BlueprintReadWrite, Category = "Planning")
TObjectPtr<AATUnit> SelectedUnit;

UPROPERTY(BlueprintReadWrite, Category = "Planning")
int32 SelectedAbilityIndex = INDEX_NONE;

```

1. Crea `WBP_AbilityButton` con:

2. icona;

3. nome;

4. testo cooldown;

5. Button.

6. Crea `WBP_AbilityBar`.

7. Quando cambia l'unità selezionata:

8. svuota la barra;

9. scorri l'array `Abilities`;

10. crea un pulsante per ogni abilità;

11. assegna indice, nome e icona.

Catena Blueprint:

```

Ability Button OnClicked
→ Set SelectedAbilityIndex
→ Get SelectedUnit.GetAbility

```

- mostra range e fase
- abilita Target Preview

1. Durante il mouse hover sulla griglia:

WorldToCell
→ distanza Manhattan dalla unità
→ confronta con RangeCells
→ verde se valido, rosso se non valido

Formula:

```
static int32 ManhattanDistance(  
    FGridCoord A,  
    FGridCoord B  
)  
{  
    return FMath::Abs(A.X - B.X)  
        + FMath::Abs(A.Y - B.Y);  
}
```

1. Quando si clicca una cella valida, costruisci o aggiorna il piano locale.
2. Il pulsante **Lock In** imposta `bLocked = true`.
3. Durante questa lezione il piano resta locale; l'RPC arriverà nella lezione multiplayer.

Asset setup

Crea materiali preview:

MI_TargetValid
MI_TargetInvalid
MI_MovePreview

Usa un Decal o un Plane leggermente sopra il terreno per evitare z-fighting.

Esercizio

Implementa `Esc`:

Se esiste un target → cancella il target
Altrimenti se esiste un'abilità → deseleziona abilità
Altrimenti → deseleziona unità

Criterio di completamento

Il giocatore può scegliere una unità, un'abilità e un bersaglio, correggere la scelta e bloccare il piano.

Lezione sette — Resolver simultaneo e conflitti

Voce	Contenuto
Obiettivo	Risolvere le intenzioni senza dipendere dall'ordine visivo
Tempo	Sette ore
Input	Piani locali e fasi
Output	Resolver a batch e conflitti di movimento
File modificati	<code>ATTurnResolver.*</code> , <code>ATTypes.h</code> , <code>ATGameMode.*</code>
Comandi chiave	Breakpoint, Output Log, <code>ensure</code>

Passi

1. Aggiungi:

```
USTRUCT()
struct FATPendingDamage
{
    GENERATED_BODY()

    UPROPERTY()
    TObjectPtr<AATUnit> Source = nullptr;

    UPROPERTY()
    TObjectPtr<AATUnit> Target = nullptr;

    UPROPERTY()
    int32 Amount = 0;
};
```

1. Crea `UATTurnResolver` basato su `UObject`.
2. Aggiungi una funzione:

```
void ResolveBlast(
    const TArray<FATActionPlan>& Plans
);
```

1. Implementa il pattern raccogli-poi-applica:

```
void UATTurnResolver::ResolveBlast(
    const TArray<FATActionPlan>& Plans
)
{
    TArray<FATPendingDamage> PendingDamage;

    for (const FATActionPlan& Plan : Plans)
```

```

{
    if (!IsValid(Plan.Unit)
        || !IsValid(Plan.TargetUnit))
    {
        continue;
    }

    UATAbilityData* Ability =
        Plan.Unit->GetAbility(Plan.AbilityIndex);

    if (!Ability
        || Ability->Phase != EATActionPhase::Blast)
    {
        continue;
    }

    FATPendingDamage Effect;
    Effect.Source = Plan.Unit;
    Effect.Target = Plan.TargetUnit;
    Effect.Amount = Ability->Power;
    PendingDamage.Add(Effect);
}

for (const FATPendingDamage& Effect : PendingDamage)
{
    if (IsValid(Effect.Target))
    {
        Effect.Target->ApplyTacticalDamage(
            Effect.Amount
        );
    }
}
}

```

1. Per Dash e Move, prima conta le destinazioni:

```

TMap<FGridCoord, int32> DestinationCounts;

for (const FATActionPlan& Plan : Plans)
{
    DestinationCounts.FindOrAdd(
        Plan.TargetCell
    )++;
}

```

1. Applica lo spostamento soltanto se:

```
DestinationCounts[Plan.TargetCell] == 1
```

1. Prima di ogni fase crea uno snapshot delle posizioni:

```
TMap<TObjectPtr<AATUnit>, FGridCoord> PhaseStartCells;
```

Tutte le validazioni della fase devono usare lo snapshot, non le posizioni che vengono animate progressivamente.

1. Nel GameMode richiama il resolver quando avanza la fase:

```
case EATMatchPhase::Blast:  
    Resolver->ResolveBlast(SubmittedPlans);  
    State->SetPhase(EATMatchPhase::Move);  
    break;
```

Regola fondamentale

Non usare `Delay` nel resolver per stabilire l'ordine delle regole. `Delay`, timeline e montage appartengono alla presentazione. Il modello tattico deve produrre il risultato immediatamente e in modo ripetibile.

Esercizio

Prepara questo test manuale:

```
A: 20 HP, spara a B  
B: 20 HP, spara ad A  
Danno: 30  
Risultato atteso: entrambe a zero HP
```

Criterio di completamento

Invertendo l'ordine dei piani nell'array, il risultato finale rimane invariato.

Lezione otto — Danno, scudi, linea di vista e feedback

Voce	Contenuto
Obiettivo	Rendere il combattimento leggibile e verificabile
Tempo	Sei ore
Input	Resolver a batch
Output	Danno, scudo, cover semplice, VFX e barra salute
File modificati	<code>ATUnit.*</code> , <code>ATCombatMath.h</code> , <code>WBP_UnitHealth</code> , materiali e VFX

Voce	Contenuto
Comandi chiave	<code>DrawDebugLine</code> , collision channel Visibility

La guida di *Atlas Reactor* includeva cover, linea di vista e ostacoli con differenti proprietà. Il vertical slice ne adotta una versione molto ridotta: gli ostacoli alti bloccano il tiro, quelli bassi possono ridurre il danno. ¹⁸

Passi

1. In `AATUnit` aggiungi:

```
UPROPERTY(
    ReplicatedUsing = OnRep_Health,
    BlueprintReadOnly
)
int32 Health = 100;

UPROPERTY(
    ReplicatedUsing = OnRep_Shield,
    BlueprintReadOnly
)
int32 Shield = 0;

UFUNCTION()
void OnRep_Health();

UFUNCTION()
void OnRep_Shield();

void ApplyTacticalDamage(int32 RawDamage);

void AddShield(int32 Amount);
```

1. Implementa:

```
void AATUnit::ApplyTacticalDamage(int32 RawDamage)
{
    if (!HasAuthority())
    {
        return;
    }

    int32 RemainingDamage = FMath::Max(0, RawDamage);

    const int32 Absorbed =
        FMath::Min(Shield, RemainingDamage);

    Shield -= Absorbed;
    RemainingDamage -= Absorbed;
```

```

        Health = FMath::Clamp(
            Health - RemainingDamage,
            0,
            MaxHealth
        );

        OnRep_Shield();
        OnRep_Health();
    }

    void AATUnit::AddShield(int32 Amount)
    {
        if (HasAuthority())
        {
            Shield = FMath::Max(0, Shield + Amount);
            OnRep_Shield();
        }
    }
}

```

1. Registra entrambe le proprietà:

```

DOREPLIFETIME(AATUnit, Health);
DOREPLIFETIME(AATUnit, Shield);

```

1. Implementa la linea di vista:

```

bool AATUnit::HasLineOfSightTo(
    const AATUnit* Target
) const
{
    if (!Target)
    {
        return false;
    }

    FHitResult Hit;
    FCollisionQueryParams Params;
    Params.AddIgnoredActor(this);

    const FVector Start =
        GetActorLocation() + FVector(0, 0, 80);

    const FVector End =
        Target->GetActorLocation()
        + FVector(0, 0, 80);

    const bool bBlocked =
        GetWorld()->LineTraceSingleByChannel(
            Hit,

```

```

        Start,
        End,
        ECC_Visibility,
        Params
    );

    return !bBlocked || Hit.GetActor() == Target;
}

```

1. Aggiungi eventi cosmetici:

```

UFUNCTION(BlueprintImplementableEvent)
void PlayHitFX(int32 Damage);

UFUNCTION(BlueprintImplementableEvent)
void PlayShieldFX(int32 Amount);

UFUNCTION(BlueprintImplementableEvent)
void PlayEliminationFX();

```

1. Richiamali da `OnRep_Health` e `OnRep_Shield`, non dalla preview locale.
2. Crea `WBP_UnitHealth` con:
3. barra HP;
4. barra scudo;
5. nome unità;
6. icona squadra.
7. Aggiungi un Widget Component sopra ogni unità.
8. Per il primo VFX usa Niagara o un semplice Particle System dello Starter Content.

Blueprint per il colpo

```

Event PlayHitFX
→ Spawn System at Location
→ Play Sound at Location
→ Create Damage Number Widget
→ Timeline di 0,4 secondi
→ Destroy Widget

```

Esercizio

Aggiungi un flag `bProvidesCover` agli ostacoli. Se la linea passa vicino a un ostacolo basso, riduci il danno del venticinque per cento.

Criterio di completamento

Scudo e salute cambiano in modo corretto, gli ostacoli bloccano il tiro e il feedback visivo non altera le regole.

Lezione nove — Multiplayer server-authoritative

Voce	Contenuto
Obiettivo	Inviare, validare e replicare i piani
Tempo	Otto ore
Input	Gameplay locale completo
Output	Partita a due giocatori in PIE o LAN
File modificati	<code>ATPlayerController.*</code> , <code>ATGameMode.*</code> , <code>ATUnit.*</code> , configurazione multiplayer
Comandi chiave	Number of Players 2, Listen Server, Network Emulation

Passi

1. Nel PlayerController dichiara:

```
UFUNCTION(Server, Reliable)
void ServerSubmitPlan(
    AATUnit* Unit,
    int32 AbilityIndex,
    FGridCoord TargetCell,
    AATUnit* TargetUnit
);
```

1. Implementa:

```
void AATPlayerController::ServerSubmitPlan_Implementation(
    AATUnit* Unit,
    int32 AbilityIndex,
    FGridCoord TargetCell,
    AATUnit* TargetUnit
)
{
    AATGameMode* GameMode =
        GetWorld()->GetAuthGameMode<AATGameMode>();

    if (!GameMode)
    {
        return;
    }

    GameMode->SubmitPlan(
        this,
```

```

        Unit,
        AbilityIndex,
        TargetCell,
        TargetUnit
    );
}

```

1. Nel GameMode valida:

```

bool AATGameMode::CanSubmitPlan(
    AATPlayerController* Controller,
    AATUnit* Unit,
    int32 AbilityIndex,
    FGridCoord TargetCell
) const
{
    const AATGameState* State =
        GetGameState<AATGameState>();

    if (!Controller || !Unit || !State)
    {
        return false;
    }

    if (State->Phase != EATMatchPhase::Planning)
    {
        return false;
    }

    if (!Unit->Abilities.IsValidIndex(AbilityIndex))
    {
        return false;
    }

    if (!GridManager->IsInsideGrid(TargetCell))
    {
        return false;
    }

    return Unit->IsOwnedByController(Controller);
}

```

1. Soltanto dopo la validazione aggiungi il piano a SubmittedPlans .
2. Non replicare SubmittedPlans ai client.
3. Replica invece:
4. fase;
5. timer;

6. turno;
7. posizione effettiva;
8. salute;
9. scudo;
10. punteggio.
11. Attiva `bReplicates = true` nelle unità.
12. Per la posizione logica usa:

```
UPROPERTY(ReplicatedUsing = OnRep_GridCell)
FGridCoord GridCell;
```

1. In `OnRep_GridCell`, avvia un'interpolazione grafica verso la posizione ricevuta.

Le RPC sono chiamate unidirezionali tra client e server; per gli input critici come la conferma di un piano è sensato usare una Server RPC affidabile, mentre gli effetti cosmetici frequenti non dovrebbero diventare la fonte autorevole dello stato. ¹⁹

Test in Editor

Apri il menu accanto a Play e configura:

```
Number of Players: 2
Net Mode: Play As Listen Server
Run Under One Process: inizialmente attivo
New Editor Window: attivo
```

Verifica:

```
Client 1 controlla soltanto il Team 0
Client 2 controlla soltanto il Team 1
Entrambi vedono lo stesso turno
Un client non può modificare HP o posizione
Le intenzioni avversarie restano nascoste
```

Poi prova una simulazione di rete:

```
Packet Lag: 150 ms
Packet Loss: 2%
```

Unreal include opzioni di emulazione della rete nelle impostazioni multiplayer del Play In Editor, utili per verificare latenza e perdita di pacchetti. ²⁰

Bug da provocare intenzionalmente

Dal client prova a inviare:

```
AbilityIndex = 999
TargetCell = (999, 999)
Unità della squadra avversaria
Piano durante Blast
```

Tutte le richieste devono essere rifiutate dal server senza crash.

Esercizio

Aggiungi un messaggio owner-only:

```
Piano rifiutato: bersaglio fuori portata
```

Usa una Client RPC soltanto verso il giocatore che ha inviato la richiesta.

Criterio di completamento

Due finestre possono pianificare contemporaneamente, il server risolve una sola volta e i client ricevono lo stesso risultato.

Lezione dieci — Test, debugging, packaging e roadmap online

Voce	Contenuto
Obiettivo	Stabilizzare il prototipo e produrre una build
Tempo	Sei ore
Input	Multiplayer funzionante
Output	Test automatici, build Windows e checklist MVP
File modificati	<code>ATCombatMath.h</code> , <code>ATCombatMathTests.cpp</code> , configurazione packaging
Comandi chiave	Test Automation, BuildCookRun, Package Project

Unreal include un Automation Framework per unit test, feature test e test di sistema; i test possono essere eseguiti dall'Editor tramite la finestra Test Automation o da riga di comando. ²¹

Passi

1. Estrai la matematica del danno in una funzione pura:

```
#pragma once

#include "CoreMinimal.h"

struct FATCombatMath
{
    static int32 DamageAfterShield(
```

```

        int32 Damage,
        int32 Shield
    )
    {
        return FMath::Max(0, Damage - Shield);
    }
};

```

1. Crea `Tests/ATCombatMathTests.cpp`:

```

#if WITH_DEV_AUTOMATION_TESTS

#include "Misc/AutomationTest.h"
#include "Combat/ATCombatMath.h"

IMPLEMENT_SIMPLE_AUTOMATION_TEST(
    FATDamageAfterShieldTest,
    "ReactorTactics.Combat.DamageAfterShield",
    EAutomationTestFlags::EditorContext
    | EAutomationTestFlags::EngineFilter
)

bool FATDamageAfterShieldTest::RunTest(
    const FString& Parameters
)
{
    TestEqual(
        TEXT("Lo scudo assorbe parte del danno"),
        FATCombatMath::DamageAfterShield(30, 20),
        10
    );

    TestEqual(
        TEXT("Lo scudo può assorbire tutto"),
        FATCombatMath::DamageAfterShield(20, 50),
        0
    );

    return true;
}

#endif

```

1. Compila.
2. Apri **Tools** → **Test Automation**.
3. Cerca:

ReactorTactics.Combat.DamageAfterShield

1. Esegui il test.
2. Aggiungi test per:
3. conversione cella/mondo;
4. distanza;
5. destinazioni contese;
6. doppia eliminazione;
7. abilità fuori range;
8. piano durante la fase errata.

Workflow di debug

Per ogni bug:

```
Riproduci
→ annota il risultato atteso
→ annota il risultato reale
→ aggiungi log o breakpoint
→ verifica se il bug è nel modello o nella presentazione
→ correggi
→ aggiungi un test
→ commit
```

Macro utili:

```
check(GridManager);

ensureMsgf(
    Unit != nullptr,
    TEXT("Resolver ricevuto senza unità")
);

UE_LOG(
    LogTemp,
    Warning,
    TEXT("Piano rifiutato per %s"),
    *GetNameSafe(Unit)
);
```

Live Coding può velocizzare molte modifiche, ma dopo cambi a `USTRUCT`, `UCLASS`, proprietà replicate o macro di reflection è più sicuro chiudere l'Editor ed eseguire una compilazione completa.

Packaging

Imposta:

```
Project Settings
→ Maps & Modes
→ Game Default Map = L_Prototype

Project Settings
→ Packaging
→ Build Configuration = Development
```

Poi:

```
Platforms
→ Windows
→ Package Project
```

Comando UAT opzionale:

```
"C:\Program Files\Epic Games\UE_5.8\Engine\Build\BatchFiles\RunUAT.bat" ^
BuildCookRun ^
-project="C:\Dev\ReactorTactics\ReactorTactics.uproject" ^
-noP4 ^
-platform=Win64 ^
-clientconfig=Development ^
-build ^
-cook ^
-stage ^
-pak ^
-archive ^
-archivedirectory="C:\Dev\ReactorTacticsBuild"
```

Checklist Definition of Done

Requisito	Completato quando
Selezione	Ogni giocatore controlla soltanto le proprie unità
Planning	Tutte le unità possono ricevere un piano
Segretezza	I piani nemici non appaiono prima della risoluzione
Fasi	Prep, Dash, Blast e Move avvengono nell'ordine corretto
Simultaneità	L'ordine dell'array non cambia i risultati
Multiplayer	Due client ricevono lo stesso stato
Validazione	Input illegali vengono respinti
UI	Fase, timer, HP, scudo e punteggio sono leggibili

Requisito	Completato quando
Test	Tutti i test automatici passano
Build	La versione pacchettizzata parte su un secondo PC
Vittoria	La partita termina e mostra il risultato

Roadmap successiva

Soltanto dopo questa checklist conviene introdurre:

```

Session browser
Epic Online Services
Lobby
Dedicated server
Reconnect
Bot
Fog of war
Gameplay Ability System
Replay
Matchmaking
Persistenza account

```

Lyra è un buon riferimento per architettura modulare, networking e Gameplay Ability System; Cropout è più utile per telecamera top-down, Enhanced Input e interfacce multiplatforma. Non è consigliabile copiare interi sistemi da questi sample: vanno studiati per responsabilità specifiche. ²²

Tooling, debug e qualità

Plugin da abilitare

Plugin	Fase	Utilizzo
Enhanced Input	Subito	Mouse, tastiera e mapping context
Visual Studio Integration	Subito	Integrazione IDE
Functional Testing	Lezione dieci	Test di gameplay
Modeling Tools Editor Mode	Opzionale	Bloccare rapidamente la mappa
Niagara	Lezione otto	Effetti visivi
Common UI	Dopo l'MVP	UI multiplatforma
Gameplay Abilities	Dopo l'MVP	Abilità avanzate
Online Subsystem EOS	Dopo l'MVP	Sessioni online
Gauntlet	Avanzato	Test automatici multiplayer

Lyra usa numerosi sistemi modulari di Unreal ed è pensato come progetto di apprendimento aggiornabile; Cropout dimostra un gioco top-down con Enhanced Input e Common UI. ²³

Plugin da evitare inizialmente

Non aggiungere nella prima fase:

- framework tattici acquistati;
- plugin griglia completi;
- runtime C#;
- inventari commerciali;
- servizi backend;
- pacchetti matchmaking;
- sistemi dialogo;
- framework GAS di terze parti.

Tali strumenti potrebbero accelerare la produzione, ma nasconderebbero le parti che il corso vuole insegnare.

Debug quotidiano

Problema	Strumento
Input non ricevuto	<code>showdebug enhancedinput</code>
Proprietà non replicata	Net Role, Output Log, RepNotify
Blueprint non eseguito	Breakpoint Blueprint
Crash C++	Debugger Visual Studio
Differenza client/server	Prefisso log con <code>HasAuthority()</code>
Movimento incoerente	Disegno delle celle e snapshot
Attacco bloccato	<code>DrawDebugLine</code>
Asset non incluso	Reference Viewer e packaging log
Prestazioni	Unreal Insights
Test multiplayer	PIE con due processi

Unreal supporta Play In Editor e Simulate In Editor per iterare rapidamente senza produrre una build completa. ²⁴

Crea una categoria di log dedicata invece di usare sempre `LogTemp`:

```
// ATLog.h
DECLARE_LOG_CATEGORY_EXTERN(
    LogReactorTactics,
    Log,
```

```
    All  
};
```

```
// ATLog.cpp  
DEFINE_LOG_CATEGORY(LogReactorTactics);
```

Uso:

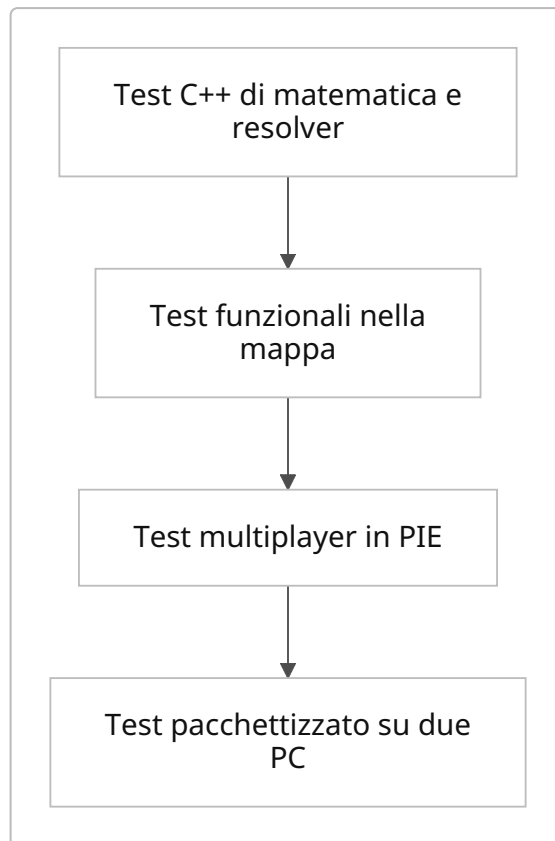
```
UE_LOG(  
    LogReactorTactics,  
    Display,  
    TEXT("Turno %d, fase %d"),  
    TurnNumber,  
    static_cast<int32>(Phase)  
);
```

Per distinguere server e client:

```
const TCHAR* NetSide =  
    HasAuthority() ? TEXT("SERVER") : TEXT("CLIENT");  
  
UE_LOG(  
    LogReactorTactics,  
    Display,  
    TEXT("[%s] HP aggiornati: %d"),  
    NetSide,  
    Health  
);
```

Strategia dei test

La piramide del progetto dovrebbe essere:



Test minimi obbligatori:

Area	Caso
Griglia	<code>CellToWorld</code> e <code>WorldToCell</code> sono coerenti
Range	Una distanza di cinque celle non passa con range quattro
Fasi	Un Blast non viene risolto durante Prep
Simultaneità	Due unità possono eliminarsi a vicenda
Movimento	Due destinazioni uguali producono conflitto
Autorità	Il client non modifica direttamente HP
Ownership	Un giocatore non controlla unità nemiche
Segretezza	Il client non riceve il piano avversario
Timeout	Il server avanza anche se un client non conferma
Packaging	Tutti gli asset richiesti vengono inclusi

Versionamento

Commit consigliati:

```
chore: setup progetto Unreal 5.8.1
feat: camera tattica e enhanced input
feat: aggiunge griglia logica
feat: aggiunge unità e ability data asset
feat: implementa stato del turno
feat: aggiunge UI di pianificazione
feat: implementa resolver simultaneo
feat: aggiunge danno e linea di vista
feat: aggiunge submit plan server RPC
test: copre conflitti e doppia eliminazione
build: configura packaging Windows
```

Prima di aggiornare Unreal crea sempre un branch e una copia del progetto. Epic raccomanda di convertire una copia, poiché un progetto salvato con una versione più recente può non essere riaperto correttamente con una versione precedente. ²⁵

Risorse e riferimenti

Installazione e primi passi

- [Download di Unreal Engine](#)
- [Installare Unreal Engine](#)
- [Specifiche hardware e software](#)
- [Configurare Visual Studio per Unreal](#)

Queste sono le fonti primarie da controllare qualora la toolchain o i requisiti cambiassero rispetto a UE 5.8.1. ²⁶

C++ per chi arriva da C

- [Scrivere codice Unreal per sviluppatori Unity, in italiano](#)
- [Programmare con C++ in Unreal](#)
- [Standard di programmazione C++ Epic](#)
- [C++ API Reference](#)

La guida per sviluppatori Unity è il ponte più diretto tra l'esperienza C# e il modello Actor, Component, UObject e reflection di Unreal. ²⁷

Blueprint, input e dati

- [Bilanciare Blueprint e C++](#)
- [Enhanced Input](#)
- [Data Assets](#)
- [Blueprint API Reference](#)

Blueprint e C++ non sono alternative esclusive: la documentazione Epic descrive esplicitamente progetti ibridi e segnala i Data Asset come soluzione semplice per dati configurabili che non richiedono istanze complesse. ³

Multiplayer

- [Networking Overview](#)
- [Actor Replication](#)
- [Remote Procedure Calls](#)
- [Replicare variabili in Blueprint](#)
- [Corso Epic: introduzione C++ al networking](#)

Replica, proprietà RepNotify e RPC costituiscono il nucleo tecnico del multiplayer del corso; la regola guida resta che lo stato di gameplay viene modificato dal server e distribuito ai client. ²⁸

Sample Epic

- [Lyra Starter Game](#)
- [Lyra su Fab](#)
- [Cropout Sample Project](#)
- [Content Examples](#)

Cropout è il riferimento più accessibile per camera top-down e input, mentre Lyra è più utile quando il corso arriva a networking, modularità e Gameplay Ability System. ²⁹

Test e debugging

- [Automation System](#)
- [Automation Technical Guide](#)
- [Live Coding](#)
- [Debug Blueprint in Visual Studio](#)
- [Source Control in Unreal](#)

L'Automation Framework copre test di basso livello e test funzionali; per scenari multiplayer più evoluti, Gauntlet può avviare sessioni con server e client multipli. ³⁰

Riferimento meccanico

- [Guida ufficiale PDF di Atlas Reactor](#)

La guida è utile esclusivamente per comprendere il principio delle decisioni simultanee, delle fasi e della comunicazione tattica. Il progetto dovrà usare nomi, personaggi, mappe, grafica, audio, lore e interfaccia originali. La struttura didattica proposta conserva il problema di game design interessante — prevedere le intenzioni dell'avversario — senza tentare di riprodurre l'intero gioco originale. ¹

¹ WELCOME, FREELANCER

https://webcdn.triongames.com/atlasreactor/AtlasReactor_BetaGuide.pdf?utm_source=chatgpt.com

² 5.8.1 Hotfix Released - Epic Developer Community Forums

https://forums.unrealengine.com/t/5-8-1-hotfix-released/2738864?utm_source=chatgpt.com

³ Balancing Blueprint and C++ | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27&utm_source=chatgpt.com

- 4 Scrivere codice in Unreal Engine per sviluppatori di Unity
https://dev.epicgames.com/documentation/unreal-engine/writing-code-in-unreal-engine-for-unity-developers?lang=it&utm_source=chatgpt.com
- 5 Hardware and Software Specifications for Unreal Engine | Unreal Engine 5.8 Documentation | Epic Developer Community
<https://dev.epicgames.com/documentation/unreal-engine/hardware-and-software-specifications-for-unreal-engine>
- 6 26 Install Unreal Engine | Unreal Engine 5.8 Documentation
https://dev.epicgames.com/documentation/unreal-engine/install-unreal-engine?utm_source=chatgpt.com
- 7 Debug Unreal Engine Blueprints in Visual Studio
https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-blueprint-debugger?utm_source=chatgpt.com
- 8 Install Visual Studio Tools for Unreal Engine
https://learn.microsoft.com/en-us/visualstudio/gamedev/unreal/get-started/vs-tools-unreal-install?utm_source=chatgpt.com
- 9 27 Programming with C++ in Unreal Engine
https://dev.epicgames.com/documentation/unreal-engine/programming-with-cplusplus-in-unreal-engine?utm_source=chatgpt.com
- 10 UnrealSharp is a plugin to Unreal Engine 5, which enables ...
https://github.com/UnrealSharp/UnrealSharp?utm_source=chatgpt.com
- 11 Gameplay Ability System | Unreal Engine 4.27 ...
https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system?application_version=4.27&utm_source=chatgpt.com
- 12 unreal.PrimaryDataAsset
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/PrimaryDataAsset?application_version=5.4&utm_source=chatgpt.com
- 13 unreal.GameModeBase
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/GameModeBase?application_version=5.5&utm_source=chatgpt.com
- 14 18 webcdn.triongames.com
https://webcdn.triongames.com/atlasreactor/AtlasReactor_BetaGuide.pdf
- 15 28 Replicating Variables in Blueprints | Unreal Engine 4.27 ...
https://dev.epicgames.com/documentation/en-us/unreal-engine/replicating-variables-in-blueprints?application_version=4.27&utm_source=chatgpt.com
- 16 Using Live Coding to recompile Unreal Engine ...
https://dev.epicgames.com/documentation/unreal-engine/using-live-coding-to-recompile-unreal-engine-applications-at-runtime?utm_source=chatgpt.com
- 17 Input | Unreal Engine 4.27 Documentation
https://dev.epicgames.com/documentation/en-us/unreal-engine/input?application_version=4.27&utm_source=chatgpt.com
- 19 Remote Procedure Calls in Unreal Engine
https://dev.epicgames.com/documentation/unreal-engine/remote-procedure-calls-in-unreal-engine?utm_source=chatgpt.com
- 20 UNetworkSettings | Unreal Engine 5.5 Documentation
https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/Engine/Engine/UNetworkSettings?application_version=5.5&utm_source=chatgpt.com

21 30 **Automation Test Framework in Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine?
utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine?utm_source=chatgpt.com)

22 23 29 **Lyra Sample Game in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine?utm_source=chatgpt.com

24 **Playing and Simulating | Unreal Engine 4.27 Documentation**

[https://dev.epicgames.com/documentation/en-us/unreal-engine/playing-and-simulating?
application_version=4.27&utm_source=chatgpt.com](https://dev.epicgames.com/documentation/en-us/unreal-engine/playing-and-simulating?application_version=4.27&utm_source=chatgpt.com)

25 **Updating Projects to Newer Versions of Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/updating-projects-to-newer-versions-of-unreal-engine?
utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/updating-projects-to-newer-versions-of-unreal-engine?utm_source=chatgpt.com)